

Department of computer science



Name: kashif khan

Class: ADCS 2A

Roll No: 2500598

Subject : database

BATCH 2025 ADCS 2A AIR UNIVERSITY, ISLAMABAD

Scenario Overview:

You are working as a database developer for BookVerse, an online bookstore. The company handles thousands of transactions daily and requires performance-optimized queries and automation through stored procedures and triggers. The system maintains records of customers, books, orders, order details, and payments.

Task 1: Stored Procedures (30 Marks)

Create and test the following stored procedures:

1. `sp_GenerateMonthlySalesReport(branch_id, month, year)` o Returns total sales, number of orders, and top-selling books for a given branch and month.
2. `sp_ApplyDiscountsToFrequentBuyers()` o Automatically applies a 10% discount to all customers who placed more than 5 orders in the past 3 months. Update a field `discount_eligible = 'Y'` in the Customers table.
3. `sp_UpdateInventoryAfterOrder(order_id)` o After a new order, reduce the book quantities in inventory according to the ordered quantity in `Order_Details`.

Task 2: Triggers (30 Marks)

Design and implement the following triggers:

1. `tr_LogBookReturns`
o After a return is processed (insert into Returns table), log the return details (customer_id, book_id, return_date, reason) into a separate `Return_Log` table.
2. `tr_RestrictNegativeInventory` o Before updating the inventory during an order, prevent the transaction if the resulting stock goes below 0. Raise an error message like "Insufficient stock."
3. `tr_AuditOrderModifications` o After any update to the Orders table, log the old and new status into an `Order_Audit` table with timestamps and the staff ID who made the change.

Task 3: Query Optimization (40 Marks)

You are given the following problem queries. For each:

- Analyze the query using EXPLAIN (or similar).
- Identify bottlenecks.
- Apply optimizations such as indexing, rewriting subqueries, or using joins.
- Show before-and-after performance metrics (costs, execution time, or row counts).

Queries to Optimize:

1. Retrieve the total sales per book category for the last 6 months.

2. List customers who placed orders for more than 5 different books in a single order.

```
• SQL CODE
•
=====
• -- BOOKVERSE DATABASE (SQL SERVER
VERSION)
•
=====
• CREATE DATABASE BookVerse; GO
• USE BookVerse;

•
•
===== --
TABLES
•
=====
•
• CREATE TABLE Customers (
customer_id INT PRIMARY KEY,
customer_name VARCHAR(100), email
VARCHAR(100), discount_eligible
CHAR(1) DEFAULT 'N'
• );
• GO
•

•
• price DECIMAL(10,2),
•
stock INT
• );
• GO
•

• CREATE TABLE Orders (
•
•
• customer_id INT,
•
branch_id INT,
• order_date DATE,
•

•
• FOREIGN KEY (customer_id)
• REFERENCES Customers(customer_id)
• );
• GO
•

• CREATE TABLE Order_Details (
•
•
•
• order id INT,
```

```

book_id INT,
quantity INT,
.
.
.
FOREIGN KEY (order_id)
REFERENCES Orders(order_id),
.
.
.
REFERENCES Books(book_id)
);
GO
.
.
CREATE TABLE Payments (
.
.
.
order_id INT,
.
.
.
payment_date DATE,
.
.
.
amount DECIMAL(10,2),
.
.
.
REFERENCES Orders(order_id)
);
GO
.
.
CREATE TABLE Returns (
.
.
.
.
.
customer_id INT,
.
.
.
book_id INT,
.
.
.
return_date DATE,
.
.
.
reason VARCHAR(255)
);
GO
.
.
CREATE TABLE Return_Log (
.
.
.
.
.
customer_id INT,
.
.
.
book_id INT,
.
.
.
return_date DATE,
.
.
.
reason VARCHAR(255)
);
GO
.
.
CREATE TABLE Order_Audit (
.
.
.
.
.
order_id INT,

```

```

    old_status VARCHAR(50),
    new_status VARCHAR(50),
    staff_id INT
);

--
=====
-- SAMPLE DATA
--
=====
INSERT INTO Customers

(1, 'Ali', 'ali@gmail.com', 'N'),
(2, 'Sara', 'sara@gmail.com', 'N');
GO

(1, 'Java Programming', 'Programming', 1000, 50),
(2, 'SQL Basics', 'Database', 800, 40);
GO

(1, 1, 1, '2026-05-01', 'Pending'),
(2, 2, 1, '2026-05-03', 'Pending');
GO

INSERT INTO Order_Details

(1, 1, 1, 2),
(2, 1, 2, 1),
(3, 2, 2, 3);
GO

(1, 1, '2026-05-10', 2800),
(2, 2, '2026-05-15', 2400);

--
===== --
STORED PROCEDURE 1
--
=====

CREATE PROCEDURE sp_GenerateMonthlySalesReport

    @p_branch_id INT,
    @p_month INT,
    @p_year INT

AS

BEGIN

```

```

•          SUM(P.amount) AS total_sales,
•          COUNT(DISTINCT O.order_id) AS total_orders
•
•
•      FROM Orders O
•
•      JOIN Payments P
•      ON O.order_id = P.order_id
•
•      WHERE O.branch_id = @p_branch_id
•      AND MONTH(O.order_date) = @p_month
•      AND YEAR(O.order_date) = @p_year;
•
•
•
•      EXEC sp_GenerateMonthlySalesReport 1,5,2026;
•      --
===== --
STORED PROCEDURE 2
•      --
=====
•
•      CREATE PROCEDURE sp_ApplyDiscountsToFrequentBuyers
•
•      AS
•
•
•
•      UPDATE Customers
•
•      SET discount_eligible = 'Y'
•
•      WHERE customer_id
IN (
•          SELECT customer_id
•
•          FROM Orders
•
•          WHERE order_date >= DATEADD(MONTH,-3,GETDATE())
•
•          GROUP BY customer_id
•
•          HAVING COUNT(order_id) > 5
•      );
•
•      END; GO
•
•      EXEC sp_ApplyDiscountsToFrequentBuyers;
•
•
•      --
===== --
STORED PROCEDURE 3

```

```

=====
•
• CREATE PROCEDURE sp_UpdateInventoryAfterOrder
•
• @p_order_id INT
•
• AS
•
•
•
• UPDATE B
•
• SET B.stock = B.stock - OD.quantity
•
• FROM Books B
•
• JOIN Order_Details OD
• ON B.book_id = OD.book_id
•
• WHERE OD.order_id = @p_order_id;
•
• END; GO
•
• EXEC sp_UpdateInventoryAfterOrder 1;

```

```

• --
===== --
TRIGGER 1

```

```

• --
=====
•
• CREATE TRIGGER tr_LogBookReturns
•
• ON Returns
•
• AFTER INSERT
•
• AS
•
•
•
• INSERT INTO Return_Log(
•
• log_id,
• customer_id,
• book_id,
• return_date,
• reason
• )
•
•
•
• SELECT
• 1,
• customer_id,
• book_id,

```

```

return_date,
reason
•         FROM inserted;
•
•     END; GO
•

•         1,
•         1,
•         1,
•         GETDATE(),
•         'Damaged Book'
•     );

•
•     --
===== --
TRIGGER 2
•     --
=====
•
•     CREATE TRIGGER tr_RestrictNegativeInventory
•
•     ON Books
•
•     INSTEAD OF UPDATE
•
•     AS
•
•     BEGIN
•
•         IF EXISTS (
•
•             FROM inserted
•             WHERE stock < 0
•         )
•
•         BEGIN
•
•             RAISERROR('Insufficient stock',16,1);
•
•         END
•
•         ELSE
•
•
•
•         UPDATE Books
•
•         SET stock = inserted.stock
•
•         FROM inserted
•
•         WHERE Books.book_id = inserted.book_id;
•

```

```

•      END
•
•
•      UPDATE Books
•      SET stock = 20
•      WHERE book_id = 1;
•
•
•      --
===== --
TRIGGER 3
•      --
=====
•
•      CREATE TRIGGER tr_AuditOrderModifications
•
•      ON Orders
•
•      AFTER UPDATE
•
•      AS
•
•
•
•      INSERT INTO Order_Audit(
•
•      audit_id,
•      order_id,
•      old_status,
•      new_status,
•      staff_id
•      )
•
•
•
SELECT
1,
•      d.order_id,
•
•      101
•
•      FROM deleted d
•
•      JOIN inserted i
•      ON d.order_id = i.order_id;
•
•
•
•      UPDATE Orders
•      SET status = 'Delivered'
•      WHERE order_id = 1;
•
•
•      --
===== --
QUERY OPTIMIZATION

```

```

=====
.
.
.  -- QUERY 1
.
.
.      B.category,
.      SUM(P.amount)
.
.  FROM Books B,
.      Order_Details OD,
.      Orders O,
.      Payments P
.
.  WHERE B.book_id = OD.book_id
.      AND OD.order_id = O.order_id
AND O.order_id = P.order_id
AND O.order_date >= DATEADD(MONTH,-6,GETDATE())
.
.  GROUP BY B.category;
.  GO
.
.  -- EXPLAIN
.
.  SET SHOWPLAN_ALL ON; GO
.
.
.      B.category,
.      SUM(P.amount)
.
.  FROM Books B,
.      Order_Details OD,
Orders O,
.      Payments P
.
.  WHERE B.book_id = OD.book_id
.      AND OD.order_id = O.order_id
.      AND O.order_id = P.order_id
.      AND O.order_date >= DATEADD(MONTH,-6,GETDATE())
.
.  GROUP BY B.category;
.  GO
.
.
.  -- INDEXES
.
.  CREATE INDEX idx_order_date
.  ON Orders(order_date);
.
.
.  CREATE INDEX idx_book_category
.  ON Books(category);
.  GO
.
.  -- OPTIMIZED QUERY

```

```

•      B.category,
•      SUM(P.amount) AS total_sales
•
•
•      FROM Orders O
•
•      JOIN Order_Details OD
•      ON O.order_id = OD.order_id
•
•      JOIN Books B
•      ON OD.book_id = B.book_id
•
•      JOIN Payments P
•      ON O.order_id = P.order_id
•
•      WHERE O.order_date >= DATEADD(MONTH,-6,GETDATE())
•
•      GROUP BY B.category;
•
•      -- QUERY 2
•
•      SELECT customer_id
•
•      FROM Orders O
•
•      JOIN Order_Details OD
•      ON O.order_id = OD.order_id
•
•      GROUP BY O.order_id, customer_id
•
•
•      CREATE INDEX idx_order_book
•      ON Order_Details(order_id, book_id);
•      GO
•
•      -- QUERY 3
•
•
•      O.order_id,
•      C.customer_name,
•      B.title,
•      P.payment_date,
•      O.order_date
•
•
•      FROM Orders O
•
•      JOIN Customers C
•      ON O.customer_id = C.customer_id
•
•
•

```

```

JOIN Order_Details OD
ON O.order_id = OD.order_id

JOIN Books B
ON OD.book_id = B.book_id

JOIN Payments P
ON O.order_id = P.order_id

WHERE DATEDIFF(DAY,O.order_date,P.payment_date) > 7;

CREATE INDEX idx_payment_date
ON Payments(payment_date);

CREATE INDEX idx_customer
ON Orders(customer_id); GO

```

Results		Messages	
	total_sales	total_orders	
1	5200.00	2	

Results		Messages					
	StmtText	StmtId	NodeId	Parent	PhysicalOp	LogicalOp	Argument
1	SELECT B.category, SUM(P.amount) FROM B...	1	1	0	NULL	NULL	1
2	--Compute Scalar(DEFINE:([Expr1004]=CASE WHE...	1	2	1	Compute Scalar	Compute Scalar	DEFINE:([Expr1004]=CASE WHEN [Expr1006]=(0) THEN ...
3	--Stream Aggregate(GROUP BY:([B].[category]) D...	1	3	2	Stream Aggregate	Aggregate	GROUP BY:([B].[category])
4	--Sort(ORDER BY:([B].[category] ASC))	1	4	3	Sort	Sort	ORDER BY:([B].[category] ASC)
5	--Nested Loops(Inner Join, OUTER REFER...	1	5	4	Nested Loops	Inner Join	OUTER REFERENCES:([OD].[book_id])
6	--Hash Match(Inner Join, HASH:([P].[order...	1	6	5	Hash Match	Inner Join	HASH:([P].[order_id])=([OD].[order_id]), RESIDUAL:([Book...
7	--Nested Loops(Inner Join, OUTER RE...	1	7	6	Nested Loops	Inner Join	OUTER REFERENCES:([P].[order_id])
8	--Clustered Index Scan(OBJECT:([B...	1	8	7	Clustered Index ...	Clustered Ind...	OBJECT:([BookVerse].[dbo].[Payments].[PK__Payments_...
9	--Clustered Index Seek(OBJECT:([B...	1	9	7	Clustered Index ...	Clustered Ind...	OBJECT:([BookVerse].[dbo].[Orders].[PK__Orders_4659...
10	--Clustered Index Scan(OBJECT:([Boo...	1	10	6	Clustered Index ...	Clustered Ind...	OBJECT:([BookVerse].[dbo].[Order_Details].[PK__Order_...
11	--Clustered Index Seek(OBJECT:([BookV...	1	11	5	Clustered Index ...	Clustered Ind...	OBJECT:([BookVerse].[dbo].[Books].[PK__Books_490D...

Defined/alias	EstimateRows	EstimateIO	EstimateCPU	AvgRowSize	TotalSubtreeCost	OutputList
NULL	2	NULL	NULL	NULL	0.04281786	NULL
[Expr1004]=CASE WHEN [Expr1006]=(0) THEN NULL ELS...	2	0	0	37	0.04281786	[B].[category], [Expr1004]
[Expr1006]=COUNT_BIG([BookVerse].[dbo].[Payments].[a...	2	0	2.8E-06	37	0.04281786	[B].[category], [Expr1006], [Expr1007]
NULL	3	0.011261...	0.00010744...	29	0.04281506	[B].[category], [P].[amount]
NULL	3	0	1.254E-05	29	0.03144635	[B].[category], [P].[amount]
NULL	3	0	0.01781149	20	0.02783451	[OD].[book_id], [P].[amount]
NULL	2	0	8.36E-06	20	0.00673472	[P].[order_id], [P].[amount]
[P].[order_id], [P].[amount]	2	0.003125	0.0001592	20	0.0032842	[P].[order_id], [P].[amount]
NULL	1	0.003125	0.0001581	10	0.0034412	NULL
[OD].[order_id], [OD].[book_id]	3	0.003125	0.0001603	15	0.0032853	[OD].[order_id], [OD].[book_id]
[B].[category]	1	0.003125	0.0001581	20	0.0035993	[B].[category]

Warnings	Type	Parallel	EstimateExecutions
NULL	SELECT	0	NULL
NULL	PLAN_ROW	0	1
NULL	PLAN_ROW	0	1
NULL	PLAN_ROW	0	1
NULL	PLAN_ROW	0	1
NULL	PLAN_ROW	0	1
NULL	PLAN_ROW	0	1
NULL	PLAN_ROW	0	1
NULL	PLAN_ROW	0	2
NULL	PLAN_ROW	0	1
NULL	PLAN_ROW	0	3

Results Messages		
	category	total_sales
1	Database	5200.00
2	Programming	2800.00

Results Messages	
	customer_id

Results Messages					
	order_id	customer_name	title	payment_date	order_date
1	1	Ali	Java Programming	2026-05-10	2026-05-01
2	1	Ali	SQL Basics	2026-05-10	2026-05-01
3	2	Sara	SQL Basics	2026-05-15	2026-05-03

